Complexity theory studies the relationship between an algorithm and the resource it takes.

What is an algorithm?

A computer algorithm is a sequence of steps a computer will take to achieve a particular outcome.

Example: Sequential Search

Given a sorted array A of length n, $(A_0 \leq A_1 \leq A_2 \leq ... \leq A_{n-1})$, and a number K, $K \in A$ sequential search is an algorithm that will go through each index in the array one by one, starting with 0, and stops when it finds the element K in the array and return it's position.

For example, if I have the array $A = [1, 5, 9, 12, 60, 110, 120, 121, 500, 505]$ and the number $K = 60$, sequential search will start checking in order:

- Is $A_0 = 60$? No, keep going.
- Is $A_1 = 60$? No, keep going.

...

- Is $A_4 = 60$? Yes, return 4, the desired position.
- This algorithm has a time complexity of $O(n)$, where $n = \text{length of the array}$, we will explain why later, but intuitively, we can think it is $O(n)$ because if the number K was at the end of the array(worst case scenario), it will take n steps to find K

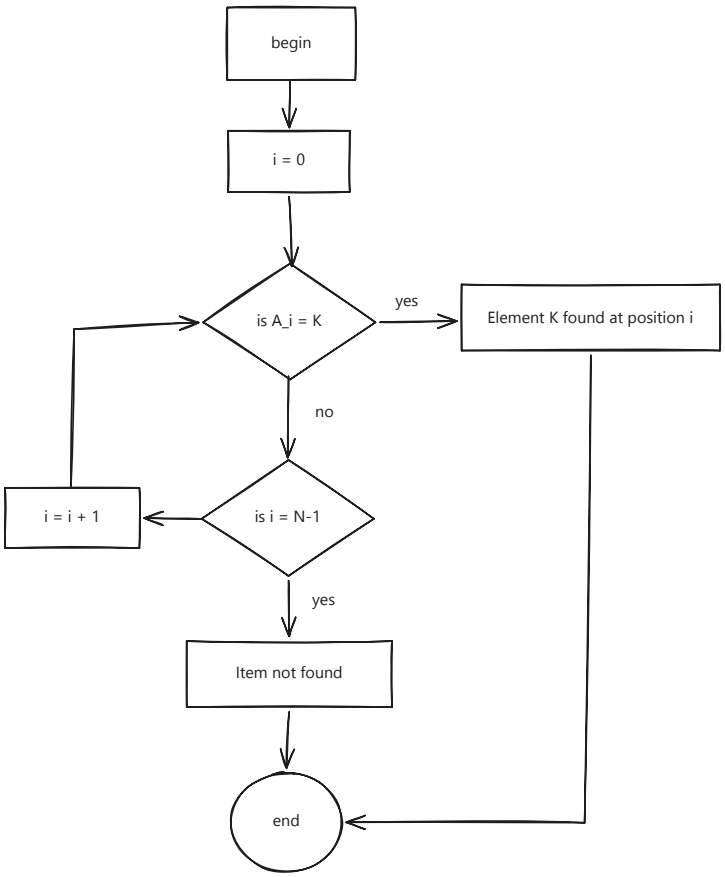


Figure 1: Sequential Search Diagram

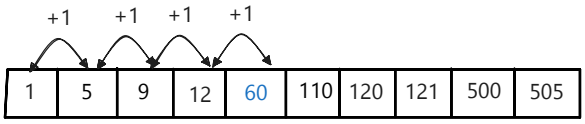


Figure 2: Sequential Search Array

Example: Binary Search

For binary search, we will consider the **sorted** 10-length array $A = [-5, -2, 0, 1, 2, 4, 5, 6, 7, 10]$ and $K = 7$, and we will start with low = 0, and high = 9, at each point, we will then repeatedly take midpoint $= \text{low} + \left\lfloor \frac{\text{high} - \text{low}}{2} \right\rfloor$, and call that point M, if $A_m > K$, then K is positioned before position m (the array is sorted), so we will take the left half, otherwise if $A_m < K$, then K is positioned after the position m, so we will take the right half, and repeat.

- This algorithm has $O(\log_2 n)$, but is written as simply $O(\log n)$ because 2 is a constant term we don't usually consider, this is because at each step in the algorithm, we cut the amount of work we have to do by half.

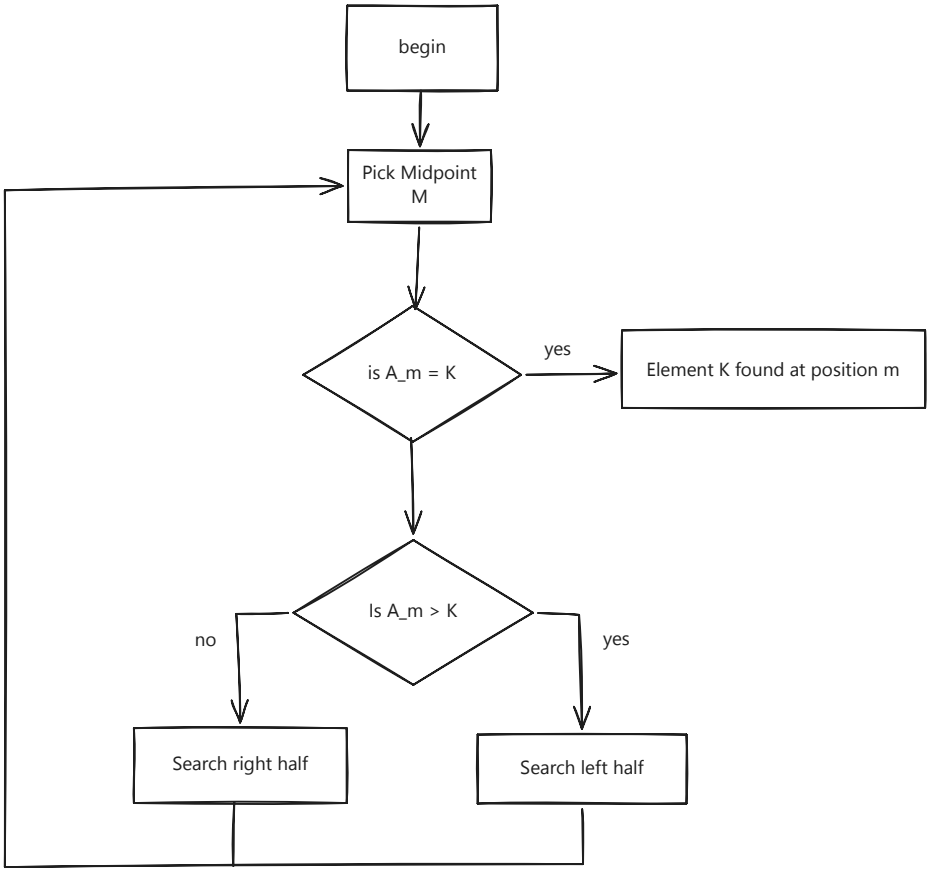


Figure 3: Binary Search Diagram

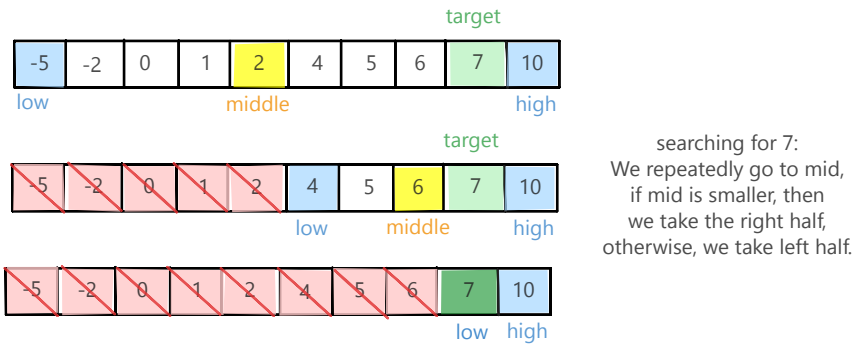


Figure 4: Binary Search Array

Time vs Space complexity?

- Time complexity describes how the execution time(number of steps) of an algorithm grows as the size of it's input grows.
- Space complexity describes how much more space an algorithm will take as the size of it's input grows.

In this course we will only be interested in time complexity.

Big-O complexity graph

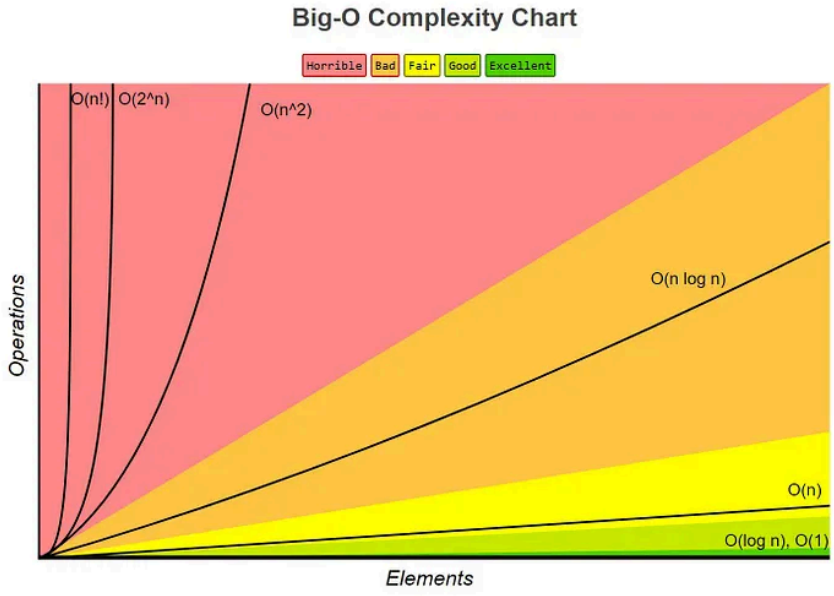


Figure 5: Big-O Complexity Chart

These are the standard time complexities we usually encounter, but we will only talk about $O(1)$, $O(n)$ and $O(n^2)$, it is important to note here that $n \geq 0$, and no negative input size exists.

Definition

$$f(n) = O(g(n)) \Leftrightarrow \exists c > 0, \exists n_0 > 0, \text{such that}$$
$$f(n) \leq c * g(n) \quad \forall n \geq n_0$$

$f(n)$ is $O(g(n))$ if there exists a $c > 0$ and an $n_0 > 0$, such that $f(n) \leq c * g(n)$ for all values of $n \geq n_0$

Explanation

Big-O describes the "upper-bound" of time an algorithm can take. Therefore, Big-O has $f(n)$ always at most $c * g(n)$ for all $n \geq n_0$.

There exists other complexity-notations:

Notation	Symbol	Meaning
Big-O	O	Upper Bound
Little-o	o	Strict upper bound
Big-Omega	Ω	Lower bound
Little-omega	ω	Strict lower bound
Big-Theta	Θ	Upper + Lower bound

The following would be true for Big-O, but not for little-o:

$$x^2 \in O(x^2)$$

$$x^2 \in O(x^2 + x)$$

$$x^2 \in O(200x^2)$$

We will only be responsible for **Big-O** in this course.

A tip: You can look at the highest degree of the function, and already be aware of what time complexity it is .

Ways of analyzing an algorithm and determine $f(n)$

Pre-requisites

- $\lfloor 5,8 \rfloor = \text{lowest integer below } 5{,}8 = 5$
- $\log_b c = a$:

$$b^a = c$$

example: $\log_2 8 = 3$

$$2^3 = 8$$

The log of base b and input c yields what exponent we need for b to equal c

- $$\log_b(b^x) = x$$
- $$\log\left(\frac{a}{b}\right) = \log a - \log b$$
- $$\log_{b_1} c = \frac{\log_{b_2} c}{\log_{b_2} b_1} = \left(\frac{1}{\log_{b_2} b_1}\right) \log_{b_2} c$$
- $$\sum_{i=1}^{k} c = c * k$$
- $$\sum_{i=1}^{k} i = 1 + 2 + 3 + ... + k = \frac{k(k+1)}{2}$$
- $$\sum_{i=1}^{n} (n-i) = (n-1) + (n-2) + (n-3) + ... + 1 + 0 = \frac{k(k-1)}{2}$$

Proof:

$$S = 1 + 2 + ... + (n-2) + (n-1)$$

$$S = (n-1) + (n-2) + ... + 2 + 1$$

Add S and the flipped S together, there is going to be $n-1$ terms in total:

$$2S = \underbrace{n + n + n + ... + n}_{n-1 \text{ terms from 1 to n}}$$

Factor out $n-1$

$$2S = n(n-1)$$

$$S = \frac{n(n-1)}{2}$$

- $$\sum_{i=1}^{k} \frac{n}{2^i} = \frac{n}{1} + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + ... + \frac{n}{2^k}$$

Analysis

First concept: In time complexity Big-O notation, we ignore constant factors and lower order terms:

$O(2n) = O(n)$, 2 is a constant factors
$O(3n^2) = O(n^2)$ 3 is a constant factors
$O(3n^2 + n) = O(n^2)$ We ignore constant factors and lower order terms
$O(\log_2 n) = O(\log n)$ Any logarithm base change differs by a constant factor(see pre-requisites)
$O(5) = O(1)$

Example algorithm 1: Algorithm 1 takes an input N, and will give me back that input N as it is, no matter the N, the algorithm will take $\text{number of steps} = 1$

It is therefore $O(1)$

Example algorithm 2: Algorithm 2 takes an array L of length N and will sum the elements of the array together, so for example, for $L = [1, 5, 2, 4, 6, 1]$, $\text{output} = 1 + 5 + 2 + 4 + 6 + 1 = 19$, it took 6 steps (length of array is $N = 6$)

The number of steps this algorithm will take is: $\text{steps} = N$, exactly N steps, each step consists of adding the number at L_k to a accumulation variable S, so the amount of work at each step is 1, $O(1)$, $S \mathrel{+}= L_k$

The amounts of steps are:

$$\sum_{i=1}^{N} 1 = N * 1 = N$$

It is therefore $O(N)$, N steps

Note: The 1 here represents the amount of work at each iteration (adding to the accumulation), which is just one step.

Example algorithm 3: Algorithm 3 takes a list L of length N, and tries to find the position of two elements (i, j), $i < j$ such that: $L[i] + L[j] = K$, where K is a target sum given.

It will do that in the following way:

- Go over each element in the list, at position i
- At position i, check all the elements infront of i, that have index j, and check if $L[i] + L[j] = K$
- If not, go to the next i

For example, if $K = 10$ and $L = [2, 5, 0, 3, 5]$

- **Starting at $i = 0$:** , We will check all elements that have position $j \geq i + 1 = 1$, but no elements would be found, because there exists no elements in this list such that $L[0] = 2$ adds with another number $L[j]$ such that $2 + L[j] = 10$
- **Trying $i = 1$:** , we would have $L[1] = 5$, then check all elements that have position $j \geq i + 1 = 2$, you would find that $L[1] + L[4] = 5 + 5 = 10 = K$, so we found it, if we didnt end up finding it, then we would try $i = 2$

The steps taken by the algorithm at the worst case are:

$$\sum_{i=0}^{N-1}(N - 1 - i) = (N - 1) + (N - 2) + ... + 1 + 0 = \frac{N(N-1)}{2} = \frac{N^2 - N}{2}$$

It is therefore $O\left(\frac{N^2-N}{2}\right)$, ignoring constant factors: $O(N^2)$

Note: It is the sum of $(N - 1 - n)$ steps because at each step i, we will end up doing $N - 1 - i$ steps, which is the length of the elements after i, when we took $i = 0$ for example, we ended up checking all $j \geq 1$, which was $N - 1$ elements, or with $i = 1$, we checked all $j \geq 2$, which was $N - 2$ elements.

Example algorithm 4: This algorithm will be Binary Search 3, we stated that at each iteration, the number of elements in binary search is cut by half, meaning that at step k, elements $= \frac{N}{2^k}$, where N is the length of the array.

Visualization: If we start with an array of length 12, after two iterations, cutting the array by half twice, there will be $\frac{12}{2^2} = 3$ elements

Since k is the number of steps, and we want to know the number of steps, and we also know that the algorithm stops when there is **1** last element:

$$\text{elements left} = 1$$

$$\frac{N}{2^k} = 1$$

$$N = 2^k$$

$$\log_2(N) = \log_2\left(2^k\right)$$

$$\log_2(N) = k$$

$$\text{steps} = k = \log_2(N)$$

It is therefore $O(\log_2 N)$, ignoring constant factor and lowest-order term:

$O(\log N)$

Extra note: log_2 N is only an integer when N is a power of 2. So the exact worst case scenario steps comes down to: $\text{steps} = \lfloor \log_2 N \rfloor + 1$

Ways of proving $f(n) = O(g(n))$

Substitution	Rebuilding
• If we want to prove $f(n) = n^2 + 5n + 3 = O(n^2)$ • We identify $g(n) = n^2$ We will need, by definition: $n^2 + 5n + 3 \leq c * g(n)$ $n^2 + 5n + 3 \leq c * n^2$ • For all $n \geq 1$, we have: $n \leq n^2$ $5n \leq 5n^2$ $3 \leq 3n^2$ • And since these are all bigger than the terms in $f(n)$, we can say: $n^2 + 5n + 3 \leq n^2 + 5n^2 + 3n^2$ $n^2 + 5n + 3 \leq 9n^2$ • So using $c = 9$ and $n_0 = 1$, we got a solution.	• If we want to prove $f(n) = n^2 + 5n + 3 = O(n^2)$ • We will take: $n_0 = 5$ $n \geq n_0$ $n \geq 5$ $5 \leq n$ • Multiplying by n, we get the middle term: $5n \leq n^2$ • For the constant term, since $n \geq 5$ we have $3 \leq n$, and $n \leq n^2$, so: $3 \leq n^2$ • And we replace: $n^2 + 5n + 3 \leq n^2 + n^2 + n^2$ $n^2 + 5n + 3 \leq 3n^2$ So using $c = 3$ and $n_0 = 5$, we got a solution.

Why is knowing the time complexity of an algorithm important?
Let's imagine a sorted array A like we did before, and let's say that array has a length of $N = 10^9$ (1 billion elements).

Using **sequential search** $O(n)$, would mean an upper-bound of operations $= o = 10^9$ operations (billion operations), to estimate how long that takes, we count CPU cycles per micro-operation:

Operation	Theoretical CPU Cycles
Load $A[i]$ from memory	1
Compare $A[i] == K$	1
Branch on result (if true stop, if false continue)	1
Increment i	1
Check if $i < n$ and branch	1

These are $c \approx 5$ cycles, and the time for the n operations in relation with clock speed and 5-cycles:

$$T(o) = \frac{\text{cycle count} * \text{operation count}}{\text{Clock Speed Frequency}} = \frac{5o}{f}$$

Where f is the clock-speed of the CPU (the speed of operation cycling) in "Hz". and o is the operation count. If we consider a theoretical CPU of clock speed 0.295 GHz (Same speed as a PlayStation 2).

$$T(o) = T(10^9) = \frac{5(10^9)}{(0.295)(10^9)} \approx 16.94 \text{ s}$$

It would take 16.94 seconds on a 0.295GHz CPU to execute the algorithm.

On the other hand, if we considered **binary search** instead with: $N = 10^9$, we would have an upper bound of:

$$o' = \log_2 N = \log_2 10^9 \approx 30$$

And so:

$$T(o') = T(\log_2 N) = \frac{5(\log_2 10^9)}{(0.295)(10^9)} \approx 5 * 10^{-7} \text{ s} \approx 0.0000005 \text{ s}$$

Comparing the two times: $\frac{T(o)}{T(o')} = \frac{16.94}{5*10^{-7}} \approx 33880000$, that's 33 million times faster. Now, realistically, there is one more operation cycle binary search does (calculating the midpoint), but I have omitted that in this calculation. This improvement in runtime cost is an **asymptotic improvement**

If I were to consider the actual CPU cycles per operations of binary search, the result would be like:

$$T_{\text{seq}}(o) = \frac{5o}{f} \qquad T_{\text{bin}}(o) = \frac{6o}{f}$$

$T_{\text{seq}}(o) = T_{\text{seq}}(10^9) = \frac{5(10^9)}{(0.295)(10^9)} \approx 16.94 \text{ s}$ $T_{\text{bin}}(o') = T_{\text{bin}}(\log_2 N) = \frac{6(\log_2 10^9)}{(0.295)(10^9)} \approx 0.0000006 \text{ s}$

(T_{bin} has 6 cycles per operation, one extra for calculating the mid-point)

Which still, isn't a big difference from the original estimation.